

Deep Learning 秋 2026

Homework 1

冯裕凯

September 21, 2026

Problem 1.1

$$\begin{aligned}\frac{\partial L}{\partial \omega_{11}^1} &= \frac{\partial L}{\partial h_3} \left(\frac{\partial h_3}{\partial h_2^1} \frac{\partial h_2^1}{\partial h_1^1} + \frac{\partial h_3}{\partial h_2^2} \frac{\partial h_2^2}{\partial h_1^1} \right) \frac{\partial h_1^1}{\partial \omega_{11}^1} \\ &= (y - \hat{y}) \sigma(z_3) (1 - \sigma(z_3)) \sigma(z_1^1) (1 - \sigma(z_1^1)) (\omega_1^3 \omega_{11}^2 \sigma(z_2^1) (1 - \sigma(z_2^1)) + \omega_2^3 \omega_{21}^2 \sigma(z_2^2) (1 - \sigma(z_2^2))) x_1\end{aligned}$$

其中, h_i^j 表示第 i 层第 j 个神经元的输出, z_i^j 表示没有激活的值。其中, z 可以继续展开, 我使用 haskell 语言基于递归编写了求导数的代码, 展开结果如下:

```
(0.5 * (((0 - ((sigmoid(((w1_3 * sigmoid(((w11_2 * sigmoid(((w11_1 * x1) + (w12_1 * x2)) + b1_1))) + (w12_2 * sigmoid(((w21_1 * x1) + (w22_1 * x2)) + b2_1))) + b1_2))) + (w2_3 * sigmoid(((w21_2 * sigmoid(((w11_1 * x1) + (w12_1 * x2)) + b1_1))) + (w22_2 * sigmoid(((w21_1 * x1) + (w22_1 * x2)) + b2_1))) + b2_2))) + b3))) * (1 - sigmoid(((w1_3 * sigmoid(((w11_2 * sigmoid(((w11_1 * x1) + (w12_1 * x2)) + b1_1))) + (w12_2 * sigmoid(((w21_1 * x1) + (w22_1 * x2)) + b2_1))) + b1_2))) + (w2_3 * sigmoid(((w21_2 * sigmoid(((w11_1 * x1) + (w12_1 * x2)) + b1_1))) + (w22_2 * sigmoid(((w21_1 * x1) + (w22_1 * x2)) + b2_1))) + b2_2))) + b3))) * ((w1_3 * ((sigmoid(((w11_2 * sigmoid(((w11_1 * x1) + (w12_1 * x2)) + b1_1))) + (w12_2 * sigmoid(((w21_1 * x1) + (w22_1 * x2)) + b2_1))) + b1_2)) * (1 - sigmoid(((w11_2 * sigmoid(((w11_1 * x1) + (w12_1 * x2)) + b1_1))) + (w12_2 * sigmoid(((w21_1 * x1) + (w22_1 * x2)) + b2_1))) + b1_2))) * (w11_2 * ((sigmoid(((w11_1 * x1) + (w12_1 * x2)) + b1_1)) * (1 - sigmoid(((w11_1 * x1) + (w12_1 * x2)) + b1_1))) * x1))) + (w2_3 * ((sigmoid(((w21_2 * sigmoid(((w11_1 * x1) + (w12_1 * x2)) + b1_1))) + (w22_2 * sigmoid(((w21_1 * x1) + (w22_1 * x2)) + b2_1))) + b2_2)) * (1 - sigmoid(((w21_2 * sigmoid(((w11_1 * x1) + (w12_1 * x2)) + b1_1))) + (w22_2 * sigmoid(((w21_1 * x1) + (w22_1 * x2)) + b2_1))) + b2_2))) * (w21_2 * ((sigmoid(((w11_1 * x1) + (w12_1 * x2)) + b1_1)) * (1 - sigmoid(((w11_1 * x1) + (w12_1 * x2)) + b1_1))) * x1)))))) * (y - sigmoid(((w1_3 * sigmoid(((w11_2 * sigmoid(((w11_1 * x1) + (w12_1 * x2)) + b1_1))) + (w12_2 * sigmoid(((w21_1 * x1) + (w22_1 * x2)) + b2_1))) + b1_2))) + (w2_3 * sigmoid(((w21_2 * sigmoid(((w11_1 * x1) + (w12_1 * x2)) + b1_1))) + (w22_2 * sigmoid(((w21_1 * x1) + (w22_1 * x2)) + b2_1))) + b2_2))) + b3))) + ((y - sigmoid(((w1_3 * sigmoid(((w11_2 * sigmoid(((w11_1 * x1) + (w12_1 * x2)) + b1_1))) + (w12_2 * sigmoid(((w21_1 * x1) + (w22_1 * x2)) + b2_1))) + b1_2))) + (w2_3 * sigmoid(((w21_2 * sigmoid(((w11_1 * x1) + (w12_1 * x2)) + b1_1))) + (w22_2 * sigmoid(((w21_1 * x1) + (w22_1 * x2)) + b2_1))) + b2_2))) + b3))) * (0 - ((sigmoid(((w1_3 * sigmoid(((w11_2 * sigmoid(((w11_1 * x1) + (w12_1 * x2)) + b1_1))) + (w12_2 * sigmoid(((w21_1 * x1) + (w22_1 * x2)) + b2_1))) + b1_2))) + (w2_3 * sigmoid(((w21_2 * sigmoid(((w11_1 * x1) + (w12_1 * x2)) + b1_1))) + (w22_2 * sigmoid(((w21_1 * x1) + (w22_1 * x2)) + b2_1))) + b2_2))) + b3))) * (1 - sigmoid(((w1_3 * sigmoid(((w11_2 * sigmoid(((w11_1 * x1) + (w12_1 * x2)) + b1_1))) + (w12_2 * sigmoid(((w21_1 * x1) + (w22_1 * x2)) + b2_1))) + b1_2))) + (w2_3 * sigmoid(((w21_2 * sigmoid(((w11_1 * x1) + (w12_1 * x2)) + b1_1))) + (w22_2 * sigmoid(((w21_1 * x1) + (w22_1 * x2)) + b2_1))) + b2_2))) + b3))) * ((w1_3 * ((sigmoid(((w11_2 * sigmoid(((w11_1 * x1) + (w12_1 * x2)) + b1_1))) + (w12_2 * sigmoid(((w21_1 * x1) + (w22_1 * x2)) + b2_1))) + b1_2)) * (1 - sigmoid(((w11_2 * sigmoid(((w11_1 * x1) + (w12_1 * x2)) + b1_1))) + (w12_2 * sigmoid(((w21_1 * x1) + (w22_1 * x2)) + b2_1))) + b1_2))) * (w11_2 * ((sigmoid(((w11_1 * x1) + (w12_1 * x2)) + b1_1)) * (1 - sigmoid(((w11_1 * x1) + (w12_1 * x2)) + b1_1))) * x1)))))) * (w11_2 *
```

$$((\text{sigmoid}(((w11_1 * x1) + (w12_1 * x2)) + b1_1)) * (1 - \text{sigmoid}(((w11_1 * x1) + (w12_1 * x2)) + b1_1)))) * x1)))) + (w2_3 * ((\text{sigmoid}(((w21_2 * \text{sigmoid}(((w11_1 * x1) + (w12_1 * x2)) + b1_1))) + (w22_2 * \text{sigmoid}(((w21_1 * x1) + (w22_1 * x2)) + b2_1)))) + b2_2)) * (1 - \text{sigmoid}(((w21_2 * \text{sigmoid}(((w11_1 * x1) + (w12_1 * x2)) + b1_1))) + (w22_2 * \text{sigmoid}(((w21_1 * x1) + (w22_1 * x2)) + b2_1)))) + b2_2)))) * (w21_2 * ((\text{sigmoid}(((w11_1 * x1) + (w12_1 * x2)) + b1_1)) * (1 - \text{sigmoid}(((w11_1 * x1) + (w12_1 * x2)) + b1_1)))) * x1))))))$$

代码如下:

```

1 data Expr = Var String
2           | Const Double
3           | Add Expr Expr
4           | Sub Expr Expr
5           | Mul Expr Expr
6           | Sigmoid Expr
7           deriving (Show,Eq)
8
9 diff :: String -> Expr -> Expr
10 diff x (Var v)
11     | x==v = Const 1
12     | otherwise = Const 0
13
14 diff _ (Const _) = Const 0
15 diff x (Add a b) = Add (diff x a) (diff x b)
16 diff x (Sub a b) = Sub (diff x a) (diff x b)
17 diff x (Mul a b) = Add (Mul (diff x a) b) (Mul a (diff x
18 b))
19 diff x (Sigmoid a) = Mul (Mul (Sigmoid a) (Sub (Const 1)
20 (Sigmoid a))) (diff x a)
21
22 v :: String -> Expr
23 v = Var
24
25 -- 第一层
26 z11 :: Expr
27 z11 =
28     Add
29     (Add
30      (Mul (v "w11_1") (v "x1"))
31      (Mul (v "w12_1") (v "x2")))
32      (v "b1_1")
33
34 z21 :: Expr
35 z21 =
36     Add
37     (Add
38      (Mul (v "w21_1") (v "x1"))
39      (Mul (v "w22_1") (v "x2")))
40      (v "b2_1")
41
42 h11 :: Expr
43 h11 = Sigmoid z11
44
45 h21 :: Expr

```

```

44 h21 = Sigmoid z21
45
46
47 -- 第二层
48 z12 :: Expr
49 z12 =
50     Add
51     (Add
52      (Mul (v "w11_2") h11)
53      (Mul (v "w12_2") h21))
54     (v "b1_2")
55
56 z22 :: Expr
57 z22 =
58     Add
59     (Add
60      (Mul (v "w21_2") h11)
61      (Mul (v "w22_2") h21))
62     (v "b2_2")
63
64 h12 :: Expr
65 h12 = Sigmoid z12
66
67 h22 :: Expr
68 h22 = Sigmoid z22
69
70
71 -- 输出层
72 z3 :: Expr
73 z3 =
74     Add
75     (Add
76      (Mul (v "w1_3") h12)
77      (Mul (v "w2_3") h22))
78     (v "b3")
79
80 yhat :: Expr
81 yhat = Sigmoid z3
82
83
84 -- 损失函数
85 loss :: Expr
86 loss =
87     Mul
88     (Const 0.5)
89     (Mul
90      (Sub (v "y") yhat)
91      (Sub (v "y") yhat))
92
93 simplify :: Expr -> Expr
94 simplify (Add a b) = case (simplify a, simplify b) of
95     (Const 0 ,b ) -> b
96     (a, Const 0 ) -> a
97     (a,b) -> Add a b

```

```

98 simplify (Sub a b) | a==b = Const 0
99                               | otherwise = case (simplify a,simplify
b) of
100                               (a,Const 0)->a
101                               (x,y)->Sub x y
102 simplify (Mul a b) = case (simplify a, simplify b) of
103                               (Const 0 , _) -> Const 0
104                               (_,Const 0) -> Const 0
105                               (Const 1 , b) -> b
106                               (a,Const 1) -> a
107                               (a,b) -> Mul a b
108 simplify (Sigmoid a) = Sigmoid (simplify a)
109
110 simplify a = a
111
112 pretty :: Expr -> String
113
114 pretty (Var x) =
115     x
116
117 pretty (Const x)
118     | x == 0 = "0"
119     | x == 1 = "1"
120     | otherwise = show x
121
122 pretty (Add a b) =
123     "(" ++ pretty a ++ " + " ++ pretty b ++ ")"
124
125 pretty (Sub a b) =
126     "(" ++ pretty a ++ " - " ++ pretty b ++ ")"
127
128 pretty (Mul a b) =
129     "(" ++ pretty a ++ " * " ++ pretty b ++ ")"
130
131 pretty (Sigmoid x) =
132     "sigmoid(" ++ pretty x ++ ")"
133
134 main = print $ pretty $ simplify $ diff "w1l_1" loss
135 --main = print $ pretty $ simplify $ diff "x" $ Mul ( v
"x" ) (Sub (v "x") (v "y"))

```

Problem 1.2

五个激活函数的一阶导数，其中 $\gamma = 0.2$ ：

$$\sigma(x) = \frac{1}{1 + e^{-x}}, \quad \sigma'(x) = \sigma(x)(1 - \sigma(x)), \quad \sigma'(0) = 0.25$$

$$\tanh(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}}, \quad \tanh'(x) = 1 - \tanh^2 x, \quad \tanh'(0) = 1$$

$$\text{LeakyReLU}(x) = \begin{cases} x & x > 0 \\ \gamma x & x \leq 0 \end{cases}, \quad \text{LeakyReLU}'(x) = \begin{cases} 1 & x > 0 \\ \gamma & x < 0 \end{cases}$$

$$\text{ELU}(x) = \begin{cases} x & x > 0 \\ \gamma(e^x - 1) & x \leq 0 \end{cases}, \quad \text{ELU}'(x) = \begin{cases} 1 & x > 0 \\ \gamma e^x & x < 0 \end{cases} = \begin{cases} 1 & x > 0 \\ \text{ELU}(x) + \gamma & x < 0 \end{cases}$$

$$\text{softplus}(x) = \ln(1 + e^x), \quad \text{softplus}'(x) = \frac{1}{1 + e^{-x}} = \sigma(x)$$

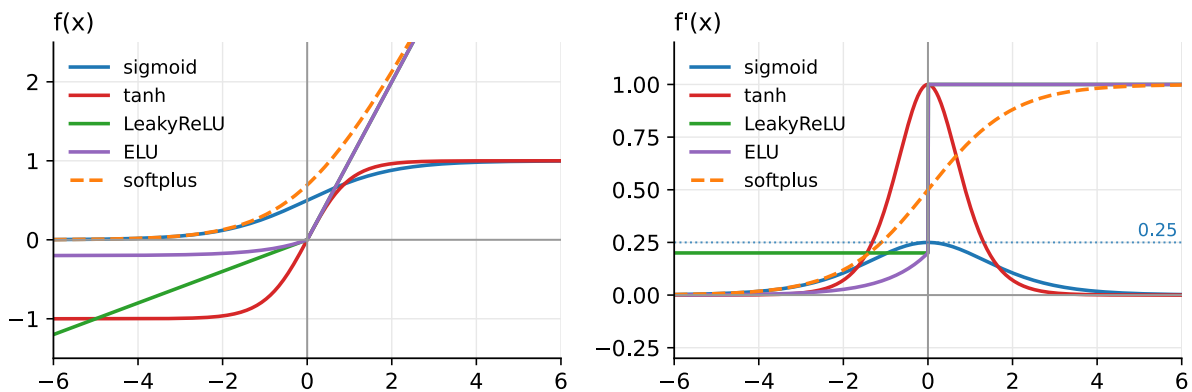


Figure 1: 左：激活函数 $f(x)$ ；右：一阶导数 $f'(x)$ 。虚线 0.25 为 sigmoid 的梯度上界。

各函数导数在截断下的区间宽，区间越宽，则梯度消失的可能性越小：

函数	$f'(0)$	$\max f'$	$f'(-3)$	$f' > 0.1$ 的区间宽
sigmoid	0.2500	0.2500	0.0452	4.13
tanh	1.0000	1.0000	0.0099	3.64
LeakyReLU	0.2 / 1	1.0000	0.2000	12.00
ELU	0.2 / 1	1.0000	0.0100	6.69
softplus	0.5000	1.0000	0.0474	8.20

其中 LeakyReLU 与 ELU 在 $x = 0$ 处不可导，表中 0.2/1 分别表示左导数与右导数。

把每层的导数看作一次梯度缩放， n 层复合后的缩放因子为 $\prod_{i=1}^n f'(x_i)$ ，取两处典型工作点 ($x = 0$ 处 LeakyReLU 与 ELU 取左导数 γ)：

函数	$[f'(0)]^{10}$	$[f'(-3)]^{10}$
sigmoid	9.5×10^{-7}	3.5×10^{-14}
tanh	1	8.7×10^{-21}
LeakyReLU	1.0×10^{-7}	1.0×10^{-7}

ELU	1.0×10^{-7}	9.6×10^{-21}
softplus	9.8×10^{-4}	5.8×10^{-14}

Problem 1.3

假设网络是全链接的，则一层输入为 m 输出为 n 的网络需要的参数为 $mn + n$, 其中 mn 为权重参数，而 n 为偏置参数 则总共需要参数

$$N = M \frac{N}{L} + \frac{N}{L} + (L-1) \left(\left(\frac{N}{L} \right)^2 + \frac{N}{L} \right) + \frac{N}{L} + 1 = -\frac{N^2}{L^2} + \frac{N^2 + (M+1)N}{L} + N + 1$$

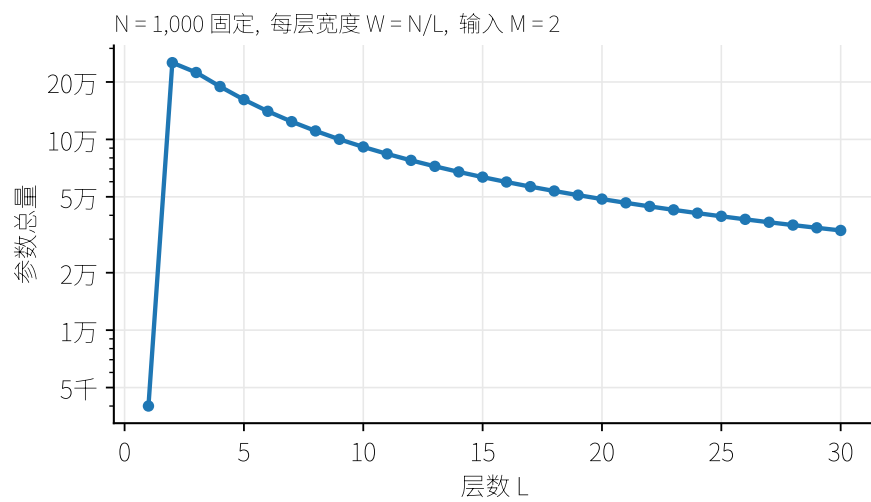


Figure 2: N 固定为 1000、输入 $M = 2$ 、每层宽度 $\frac{N}{L}$ 时，参数总量随层数 L 的变化。